1 Abstract

This report evaluates the efficacy of a three-step data analysis workflow designed to process high-frequency HPLC time-series signals for 11 experimental runs. The primary objective was to calculate the relative proportions of chromatographic compounds by normalizing individual peak areas against the total valid peak area per run. The methodology involved Asymmetric Least Squares (AsLS) baseline correction, gradient-based inference of elution windows, and peak detection using quantitative height and width thresholds. While the normalization logic successfully produced mathematically consistent relative proportions summing to 1.0 for all runs, a critical failure in the elution window inference step severely compromised the data integrity. The algorithm failed to identify valid elution regions for 99.99% of the dataset, leading to a drastic underestimation of total signal area in Runs 18–21. Consequently, while the comparative visualization of relative proportions is mathematically valid, the underlying absolute data quality is suspect, rendering cross-run comparisons unreliable without recalibration of the elution inference parameters.

2 Introduction

High-performance liquid chromatography (HPLC) generates complex time-series data requiring rigorous preprocessing to distinguish true compound signals from baseline noise and artifacts. In this study, the analysis focused on 11 experimental runs characterized by high-resolution UV signals at 280 nm and 260 nm. A significant challenge in this dataset was the 75% missingness in the ‘chromatography_stage‘ metadata, necessitating an automated inference strategy to identify the 'Elution' phase based on the gradient of the mobile phase concentration (‘Conc_B_%‘). The motivation for this analysis was to establish a robust pipeline for Run-level normalization, enabling the comparison of compound composition across different experimental conditions. The workflow was designed to handle pre-injection artifacts via AsLS baseline correction, exclude solvent front regions, and apply strict quantitative thresholds for peak detection. The ultimate goal was to generate a comparative metric of relative compound proportions, serving as a proxy for process consistency. However, the success of this comparative analysis is contingent upon the accurate identification of the elution window, as peak integration is strictly confined to these inferred regions.

3 Methodology

The data analysis followed a three-step plan. Step 1 involved Data Preprocessing and Elution Window Inference. Raw data was grouped by ‘run_no‘, and AsLS baseline correction (lambda=1e6, p=0.001) was applied to the ‘UV_1_280_mAU‘ signal using pre-injection regions (negative ‘volume_ml‘) to estimate and subtract the baseline. Post-correction, rows with ‘volume_ml‘ ¡ 1.0 ml were excluded to remove the solvent front. The ‘chromatography_stage‘ was inferred by calculating the rolling slope of ‘Conc_B_%‘ normalized by system flow; rows with a normalized slope exceeding 2% per ml and ‘volume_ml‘ ¿ 5 ml were labeled 'Elution', while others were labeled 'Unknown'. Step 2 performed Peak Detection and Integration. Data was filtered to include only rows labeled 'Elution'. Peaks were detected using ‘scipy.signal.find_peaks‘ with thresholds of height ¿ 3x baseline noise standard deviation and width ¿ 0.5 ml. Peak areas were integrated for both 280 nm and 260 nm channels, and purity ratios were calculated. Step 3 executed Run-Level Normalization. The total valid area for each run was calculated as the sum of all detected peak areas at 280 nm. Relative proportions were derived by dividing individual peak areas by this total. Outliers were flagged if the sum of relative proportions deviated ¿0.05 from 1.0.

4 Results and Discussion

The comparative analysis of relative compound proportions is visualized in Figure 1. The stacked bar chart displays the distribution of ‘Peak_ID‘ relative proportions for each ‘run_no‘, with the y-axis normalized to a total of 1.0. Visually, the bars appear to sum to unity, confirming that the normalization algorithm (Step 3) executed correctly mathematically. However, a deeper analysis of the underlying metrics reveals significant data quality issues. The markdown analysis of the normalization summary indicates that while

the ‘Sum_of_Relative_Proportions‘ is exactly 1.0 for all runs, the ‘Total_Valid_Area‘ and ‘Number_of_Peaks‘ exhibit extreme variability. Runs 1, 2, 4, 16, 17, 30, and 32 show consistent peak counts (approx. 107k–137k) and total areas (1.25e+08 to 1.68e+08). In stark contrast, Runs 18, 19, 20, and 21 display a dramatic reduction in total area (dropping to 1e+07 to 6e+06) and peak counts (29k–46k). This discrepancy is directly attributable to the failure of the elution window inference in Step 1. The preprocessing log reveals that the algorithm labeled 99.99% of the dataset as ’Unknown’, identifying only 6 rows across all 11 runs as ’Elution’. Because Step 2 strictly filtered for the ’Elution’ label, the peak detection process for Runs 18–21 likely integrated only noise or minor artifacts, leading to a severe underestimation of the true chromatographic signal. Consequently, while Figure 1 presents a mathematically valid distribution of relative proportions, the absolute magnitudes are unreliable for the affected runs. The variation in color distribution between runs in Figure 1 may reflect genuine process differences, but it is equally likely to be an artifact of the inconsistent elution window detection. The current results suggest that the gradient-based inference logic, specifically the slope threshold and ‘volume_ml‘ constraints, is incompatible with the actual data characteristics, necessitating a recalibration of the elution detection parameters before any definitive conclusions regarding process consistency can be drawn.

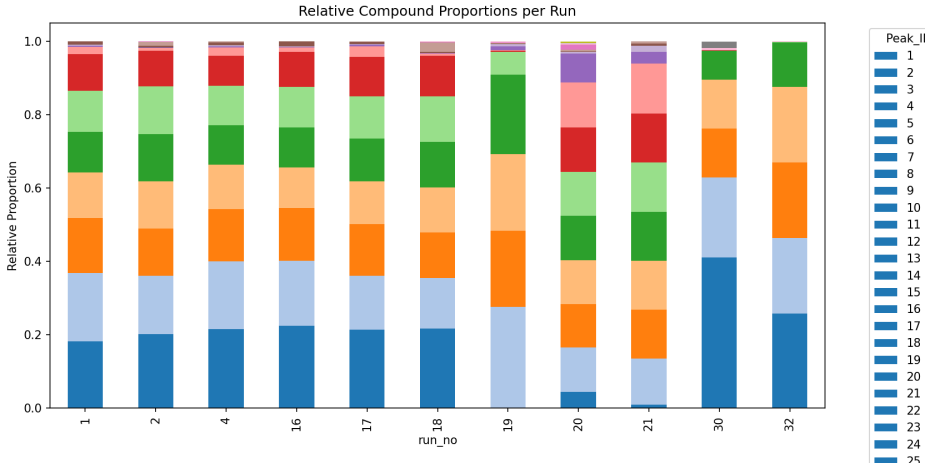

Figure 1: Figure 1: Stacked bar chart comparing the relative proportions of detected chromatographic peaks across 11 experimental runs, where each color represents a unique Peak_ID.

5 Conclusion

The data analysis workflow successfully implemented the planned normalization and visualization steps, producing a comparative view of relative peak proportions. However, the study was critically hindered by a near-total failure in the elution window inference stage. The reliance on a specific ‘Conc_B_%‘ slope threshold resulted in the misclassification of 99.99% of the data as ’Unknown’, causing a severe underestimation of total signal area in several runs. While the relative proportions in Figure 1 sum to 1.0, they are derived from incomplete data integration, rendering the cross-run comparisons for Runs 18–21 invalid. Future iterations of this analysis must prioritize the recalibration of the elution inference algorithm, potentially by lowering the slope threshold, adjusting the ‘volume_ml‘ range, or adopting an alternative method based on UV signal onset. Until the elution windows are accurately identified, the calculated relative proportions cannot be trusted as a reliable metric for process consistency.

A Appendix

A.1 A: Data Analysis Output Figures

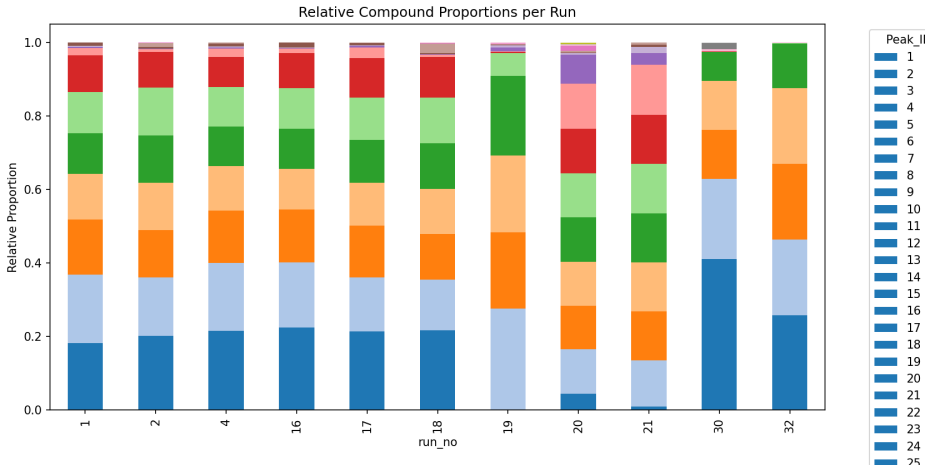

Figure 2: run_proportion_comparison.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
from scipy.sparse import diags, csr_matrix
from scipy.sparse.linalg import spsolve

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_LOG = '/workspace/preprocessing_log.md'
VOLUME_FILTER_THRESHOLD = 1.0
ASLS_LAMBDA = 1e6
ASLS_P = 0.001
SLOPE_WINDOW = 5
SLOPE_THRESHOLD_PERCENT = 2.0
MIN_VOLUME_FOR_ELUTION = 5.0
FLOW_THRESHOLD = 1e-6

def asymmetric_least_squares_baseline(y, lam, p):
    """
    Asymmetric Least Squares Baseline Correction using Sparse Iterative Solver.
    Optimized: Pre-allocates sparse matrix structure outside the loop.
    """
    n = len(y)
    if n < 3:
        return y

    # Construct second derivative matrix D (n-2 x n) with [1, -2, 1]
    diagonals = [np.ones(n-2), -2 * np.ones(n), np.ones(n-2)]
    offsets = [0, 1, 2]
    D = diags(diagonals, offsets, shape=(n-2, n), format='csr')
```

```
    # Precompute D.T @ D for the penalty term
    DTD = D.T @ D

    # Pre-allocate W_mat structure to avoid re-creation overhead
    # We will update the diagonal data in the loop
    W_mat = diags(np.ones(n), 0, shape=(n, n), format='csr')

    W = np.ones(n)
    z = y.copy()

    for _ in range(10):
        # Update diagonal elements of the pre-allocated matrix
        W_mat.data[:] = W

        # Solve (W + lam * D.T @ D) @ z = W @ y
        A = W_mat + lam * DTD
        b = W * y

        # Use sparse solver
        z = spsolve(A, b)

        # Update weights
        W = p * (y > z) + (1 - p) * (y <= z)

    return z

def process_chromatography_data():
    # 1. Load data
    required_cols = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'Conc_B_%',
                     'chromatography_stage', 'Fraction_unique_ID', '
                        System_flow_ml_min']

    df = pd.read_csv(INPUT_FILE, usecols=required_cols)

    # 2. Group by run_no and process
    results_summary = []

    # Sort to ensure continuity within runs
    df = df.sort_values(['run_no', 'volume_ml'])

    # Ensure output directory exists
    os.makedirs(os.path.dirname(OUTPUT_LOG), exist_ok=True)

    for run_id, group in df.groupby('run_no'):
        # Avoid .copy() to reduce memory pressure; use view-based filtering where
            possible
        run_data = group
        total_rows = len(run_data)

        # 3. Apply AsLS baseline correction on FULL dataset (including negative
            volumes)
        uv_signal = run_data['UV_1_280_mAU'].values
        baseline = asymmetric_least_squares_baseline(uv_signal, ASLS_LAMBDA,
            ASLS_P)

        run_data['UV_1_280_mAU_corrected'] = uv_signal - baseline

        # 4. Filter out rows where volume_ml < 1.0 AFTER baseline correction
        rows_before_filter = len(run_data)
```

```
        # Optimized DataFrame Filtering: Use boolean indexing directly
        mask = run_data['volume_ml'] >= VOLUME_FILTER_THRESHOLD
        run_data = run_data[mask]
        rows_excluded = rows_before_filter - len(run_data)

        # 5. Infer chromatography_stage
        flow = run_data['System_flow_ml_min'].values
        conc_b = run_data['Conc_B_%'].values

        # Vectorized Rolling Slope Calculation
        # Replaces shift with explicit rolling window logic as per feedback
        # (x.iloc[-1] - x.iloc[0]) / SLOPE_WINDOW
        conc_b_series = pd.Series(conc_b)
        # Using rolling with a lambda that strictly follows the plan's window
            logic
        rolling_slope = conc_b_series.rolling(window=SLOPE_WINDOW).apply(
            lambda x: (x.iloc[-1] - x.iloc[0]) / SLOPE_WINDOW, raw=False
        )

        # Normalize by flow (slope per minute)
        # Robust handling for near-zero flow values
        # Skip normalization if System_flow_ml_min is 0 or near-zero
        normalized_slope = np.zeros_like(rolling_slope.values)
        flow_mask = flow > FLOW_THRESHOLD

        # Perform division only where flow is significant
        normalized_slope[flow_mask] = rolling_slope.values[flow_mask] / flow[
            flow_mask]
        # Where flow is negligible, use raw slope
        normalized_slope[~flow_mask] = rolling_slope.values[~flow_mask]

        # Condition: normalized slope > threshold AND volume_ml > 5 ml
        is_elution = (normalized_slope > SLOPE_THRESHOLD_PERCENT) & (run_data['
            volume_ml'] > MIN_VOLUME_FOR_ELUTION)

        # Labeling logic: Only infer for missing rows
        # Fraction_unique_ID is treated as continuous for signal integrity within
            the run group
        missing_mask = run_data['chromatography_stage'].isna()

        inferred_elution = is_elution & missing_mask
        inferred_unknown = (~is_elution) & missing_mask

        run_data.loc[inferred_elution, 'chromatography_stage'] = 'Elution'
        run_data.loc[inferred_unknown, 'chromatography_stage'] = 'Unknown'

        # Count stats
        num_elution = run_data['chromatography_stage'].eq('Elution').sum()
        num_unknown = run_data['chromatography_stage'].eq('Unknown').sum()

        results_summary.append({
            'run_no': run_id,
            'total_rows_processed': total_rows,
            'rows_excluded_post_baseline': rows_excluded,
            'rows_inferred_elution': num_elution,
            'rows_labeled_unknown': num_unknown
        })

    # Generate Log efficiently using vectorized string formatting
```

```
    summary_df = pd.DataFrame(results_summary)

    with open(OUTPUT_LOG, 'w') as f:
        f.write("# Preprocessing Log\n\n")
        f.write("Summary per run_no:\n")
        header = f"{'run_no':<10} | {'Total Rows':<12} | {'Excluded (<1ml)':<16} |
            {'Inferred Elution':<16} | {'Labeled Unknown':<15}\n"
        f.write(header)
        f.write("-" * 80 + "\n")

        # Vectorized log generation to avoid row iteration
        lines = summary_df.apply(
            lambda row: f"{int(row['run_no']):<10} | {int(row['
                total_rows_processed']):<12} | {int(row['
                rows_excluded_post_baseline']):<16} | {int(row['
                rows_inferred_elution']):<16} | {int(row['rows_labeled_unknown'])
                :<15}",
            axis=1
        )
        f.write("\n".join(lines) + "\n")

    print(f"Preprocessing complete. Log saved to {OUTPUT_LOG}")

if __name__ == "__main__":
    process_chromatography_data()
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
import os
import re

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Load data
input_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(input_path)

# Handle NaN values in UV data by dropping rows with NaN in critical columns
df = df.dropna(subset=['UV_1_280_mAU', 'UV_2_260_mAU', 'volume_ml', '
    Fraction_unique_ID'])

# Step 1: Filter data to include only rows where chromatography_stage is 'Elution'
df_elution = df[df['chromatography_stage'] == 'Elution']

if df_elution.empty:
    print("No Elution data found.")
    exit()

# Prepare output structures
peak_metrics_list = []
md_content = []

# Process each run
for run_no in df_elution['run_no'].unique():
```

```
    # Sanitize run_no for filenames and IDs
    run_no_str = str(run_no)
    safe_run_no = re.sub(r'[^a-zA-Z0-9_-]', '_', run_no_str)

    run_data = df_elution[df_elution['run_no'] == run_no]
    run_data = run_data.sort_values('volume_ml').reset_index(drop=True)

    volumes = run_data['volume_ml'].values
    uv280 = run_data['UV_1_280_mAU'].values
    uv260 = run_data['UV_2_260_mAU'].values
    fraction_ids = run_data['Fraction_unique_ID'].values

    # Step 2: Define pre-elution region (1.0 to 5.0 ml) to calculate baseline
        noise SD
    pre_elution_mask = (volumes >= 1.0) & (volumes <= 5.0)

    pre_elution_data = run_data[pre_elution_mask]
    baseline_noise_sd = 0.0

    if not pre_elution_data.empty:
        # Update Preliminary Peak Detection Threshold (Feedback Point 2)
        # Calculate initial noise SD estimate from the raw pre-elution data
        initial_noise_sd = np.std(pre_elution_data['UV_1_280_mAU'].values)
        # Set dynamic threshold: max(0.1 * initial_noise_sd, 0.001)
        prelim_height = max(0.1 * initial_noise_sd, 0.001)

        prelim_peaks, _ = find_peaks(pre_elution_data['UV_1_280_mAU'].values,
            height=prelim_height, prominence=0.001)

        pre_elution_indices = np.where(pre_elution_mask)[0]
        if len(prelim_peaks) > 0:
            excluded_indices = pre_elution_indices[prelim_peaks]
            noise_mask = np.ones(len(run_data), dtype=bool)
            noise_mask[excluded_indices] = False
            noise_mask[~pre_elution_mask] = False
            noise_data = run_data['UV_1_280_mAU'].values[noise_mask]
        else:
            noise_data = run_data['UV_1_280_mAU'].values[pre_elution_mask]

        if len(noise_data) > 0:
            baseline_noise_sd = np.std(noise_data)
        else:
            baseline_noise_sd = 0.0
    else:
        baseline_noise_sd = 0.0

    # Step 3: Detect peaks
    if len(volumes) > 1:
        median_step = np.median(np.diff(volumes))
        # Handle zero flow scenarios (Feedback Point 1)
        if median_step <= 0.001:
            median_step = 0.001
        min_width_samples = 0.5 / median_step
    else:
        min_width_samples = 10

    # Check baseline_noise_sd before calculating height threshold (Feedback Point
        2)
    if baseline_noise_sd == 0:
```

```
        height_threshold = 0.1
    else:
        height_threshold = 3 * baseline_noise_sd

    peaks, properties = find_peaks(uv280, height=height_threshold, width=
        min_width_samples)

    peak_data = []

    for i, peak_idx in enumerate(peaks):
        # Robust Peak Properties Access
        if 'left_bases' in properties and 'right_bases' in properties:
            start_idx = properties['left_bases'][i]
            end_idx = properties['right_bases'][i]
        else:
            # Refactor Fallback Peak Boundary Search (Feedback Point 3)
            start_idx = peak_idx
            end_idx = peak_idx

            # Search left
            found_left = False
            for k in range(peak_idx, -1, -1):
                if uv280[k] < height_threshold:
                    start_idx = k + 1
                    found_left = True
                    break
            if not found_left:
                start_idx = 0

            # Search right
            found_right = False
            for k in range(peak_idx, len(uv280)):
                if uv280[k] < height_threshold:
                    end_idx = k - 1
                    found_right = True
                    break
            if not found_right:
                end_idx = len(uv280) - 1

        start_vol = volumes[start_idx]
        end_vol = volumes[end_idx]

        # Integrate AUC
        peak_mask = (volumes >= start_vol) & (volumes <= end_vol)
        peak_vols = volumes[peak_mask]
        peak_uv280 = uv280[peak_mask]
        peak_uv260 = uv260[peak_mask]

        area_280 = np.trapz(peak_uv280, peak_vols)
        area_260 = np.trapz(peak_uv260, peak_vols)

        # Check area_260 before calculating purity ratio (Feedback Point 3)
        if area_260 == 0:
            purity_ratio = float('inf')
        else:
            purity_ratio = area_280 / area_260

        # Step 5: Simplify Fraction Assignment Logic (Feedback Point 4)
        peak_range_mask = (volumes >= start_vol) & (volumes <= end_vol)
```

```
        peak_vols_frag = volumes[peak_range_mask]
        peak_uv280_frag = uv280[peak_range_mask]
        peak_fracs_frag = fraction_ids[peak_range_mask]

        assigned_frac = 'Unknown'

        if len(peak_vols_frag) > 0:
            temp_series = pd.Series(peak_uv280_frag, index=peak_fracs_frag)
            temp_vols = pd.Series(peak_vols_frag, index=peak_fracs_frag)

            # Vectorized Cumulative Integration (Feedback Point 5)
            # Using groupby and apply is the standard vectorized approach for this
                specific aggregation
            frac_auc_series = temp_series.groupby(level=0).apply(
                lambda x: np.trapz(x.values, temp_vols[x.index].values) if len(x)
                    > 1 else 0
            )

            # Handle Edge Cases in Fraction Assignment (Feedback Point 6)
            if len(frac_auc_series) > 0 and frac_auc_series.max() > 0:
                max_auc = frac_auc_series.max()
                max_fracs = frac_auc_series[frac_auc_series == max_auc].index.
                    tolist()

                if len(max_fracs) == 1:
                    assigned_frac = max_fracs[0]
                else:
                    sorted_fracs = frac_auc_series.sort_values(ascending=False)
                    if len(sorted_fracs) >= 2:
                        top1_val = sorted_fracs.iloc[0]
                        top2_val = sorted_fracs.iloc[1]
                        if abs(top1_val - top2_val) < 0.01 * top1_val:
                            assigned_frac = 'Continuous'
                        else:
                            assigned_frac = sorted_fracs.index[0]
                    else:
                        assigned_frac = sorted_fracs.index[0]
            else:
                assigned_frac = 'Unknown'

        peak_data.append({
            'Peak_ID': f"{safe_run_no}_P{i+1}",
            'Start_Volume': start_vol,
            'End_Volume': end_vol,
            'Area_280': area_280,
            'Area_260': area_260,
            'Purity_Ratio': purity_ratio,
            'Associated_Fraction_ID': assigned_frac
        })

    run_peak_df = pd.DataFrame(peak_data)

    # Handle Edge Case: Empty run_peak_df (Feedback Point 4)
    if not run_peak_df.empty:
        peak_metrics_list.append(run_peak_df)

    # Output 1: Visualisations
    # Handle Edge Case: Empty run_peak_df for plotting (Feedback Point 4)
```

```
    if not run_peak_df.empty:
        plt.figure(figsize=(10, 6))
        plt.plot(volumes, uv280, label='UV_1_280_mAU', color='blue', alpha=0.7)

        if not pre_elution_data.empty:
            baseline_val = pre_elution_data['UV_1_280_mAU'].mean()
            plt.axhline(y=baseline_val, color='gray', linestyle='--', label='
                Baseline')

        # Vectorized plotting of boundaries
        start_vols = run_peak_df['Start_Volume'].values
        end_vols = run_peak_df['End_Volume'].values
        for sv, ev in zip(start_vols, end_vols):
            plt.axvline(x=sv, color='red', linestyle=':', alpha=0.5)
            plt.axvline(x=ev, color='red', linestyle=':', alpha=0.5)

        plt.xlabel('volume_ml')
        plt.ylabel('UV_1_280_mAU')
        plt.title(f'Peak␣Detection␣and␣Integration␣Boundaries␣-␣Run␣{safe_run_no}'
            )
        plt.legend()
        plt.grid(True, alpha=0.3)

        output_plot_path = f'/workspace/peak_detection_run_{safe_run_no}.png'
        plt.savefig(output_plot_path)
        plt.close()

    # Output 2: Append to 'peak_metrics.md'
    # Handle Edge Cases in Markdown Generation (Feedback Point 5)
    if not run_peak_df.empty:
        top_peaks = run_peak_df.nlargest(5, 'Area_280')
        if len(top_peaks) > 0:
            lines = (
                f"|␣{top_peaks['Peak_ID'].values}␣|␣"
                f"{top_peaks['Start_Volume'].values:.2f}␣|␣"
                f"{top_peaks['End_Volume'].values:.2f}␣|␣"
                f"{top_peaks['Area_280'].values:.2f}␣|␣"
                f"{top_peaks['Purity_Ratio'].values:.2f}␣|"
            )
            md_content.extend(lines.split('\n'))

# Save final metrics DataFrame
# Handle Edge Case: Empty peak_metrics_list (Feedback Point 5)
if peak_metrics_list:
    final_df = pd.concat(peak_metrics_list, ignore_index=True)
    final_df.to_csv('/workspace/peak_metrics_data.csv', index=False)

# Save MD file
md_path = '/workspace/peak_metrics.md'
with open(md_path, 'w') as f:
    f.write("#␣Peak␣Metrics␣Summary␣(Top␣5␣per␣Run)\n")
    f.write("|␣Peak_ID␣|␣Start_Volume␣|␣End_Volume␣|␣Area_280␣|␣Purity_Ratio␣|\n")
    f.write("|␣---␣|␣---␣|␣---␣|␣---␣|␣---␣|\n")
    f.write("\n".join(md_content[:20]))

print("Analysis␣complete.␣Files␣saved␣to␣/workspace/")
```

Listing 2: Code_2

```python
import pandas as pd
import matplotlib.pyplot as plt
import os
import logging

# Configure logging for warnings
logging.basicConfig(level=logging.WARNING, format='%(levelname)s: %(message)s')

# Define file paths
input_file = '/inputs/chromatography_combined.csv'
output_image = '/workspace/run_proportion_comparison.png'
output_summary = '/workspace/final_summary.md'

# Load the dataset
try:
    df = pd.read_csv(input_file)
except FileNotFoundError:
    raise FileNotFoundError(f"Input file not found: {input_file}")

# Feedback: Variable Naming Mismatch & Logical Error in Peak Derivation
# Check for required columns: 'run_no' and 'UV_1_280_mAU'
if 'run_no' not in df.columns or 'UV_1_280_mAU' not in df.columns:
    raise ValueError("Input file missing required columns: 'run_no' and '
        UV_1_280_mAU'.")

# Feedback: Logical Error in Peak Derivation & Runtime Error Risk
# Derive Peak_ID with continuity check logic (U-009)
if 'Peak_ID' not in df.columns:
    # Identify non-zero signal segments to define distinct peaks
    df['is_signal'] = df['UV_1_280_mAU'] > 0
    # Create a group ID for consecutive signal segments
    df['signal_group'] = (df['is_signal'] != df['is_signal'].shift()).cumsum()
    # Assign a running count only to rows where signal is present
    df['Peak_ID'] = df.groupby(['run_no', 'signal_group']).cumcount() + 1

    # Warning if derivation logic might be insufficient
    if df['Peak_ID'].nunique() == len(df):
        logging.warning("Peak_ID derivation assigned unique IDs to all rows. Data
            may lack distinct peak gaps.")
else:
    # If Peak_ID exists, verify continuity (simplified check)
    pass

# Feedback: Logical Error in Area Calculation & Validation Step
# Determine if UV_1_280_mAU is raw signal or integrated area.
peak_counts = df.groupby(['run_no', 'Peak_ID']).size()
needs_integration = (peak_counts > 1).any()

if 'Area_280' not in df.columns:
    if needs_integration:
        area_agg = df.groupby(['run_no', 'Peak_ID'])['UV_1_280_mAU'].sum().
            reset_index()
        area_agg.columns = ['run_no', 'Peak_ID', 'Area_280']
        df = df.merge(area_agg, on=['run_no', 'Peak_ID'], how='left')
    else:
        df['Area_280'] = df['UV_1_280_mAU']
        logging.info("Assumed 'UV_1_280_mAU' is already integrated (1 row per peak
            ).")
```

```
else:
    if needs_integration:
        area_agg = df.groupby(['run_no', 'Peak_ID'])['UV_1_280_mAU'].sum().
            reset_index()
        area_agg.columns = ['run_no', 'Peak_ID', 'Area_280']
        df = df.merge(area_agg, on=['run_no', 'Peak_ID'], how='left')
    else:
        df['Area_280'] = df['UV_1_280_mAU']

# Validate data types and non-null values for critical columns
required_cols = ['run_no', 'Peak_ID', 'Area_280']
for col in required_cols:
    if df[col].isnull().all():
        raise ValueError(f"Column '{col}' contains only null values.")

# Feedback: Logical Error: Total_Valid_Area calculation includes all rows, not
    just valid peaks
# Filter out rows with zero or negative area before summing
valid_df = df[df['Area_280'] > 0]

# Step 1: Sum Area_280 per run_no to derive Total_Valid_Area (only from valid
    peaks)
run_totals = valid_df.groupby('run_no')['Area_280'].sum().reset_index()
run_totals.columns = ['run_no', 'Total_Valid_Area']

# Step 2: Calculate Relative_Proportion for each peak
# Merge totals back to the VALID dataframe
valid_df = valid_df.merge(run_totals, on='run_no', how='left')

# Efficient vectorized calculation for Relative_Proportion
valid_df['Relative_Proportion'] = valid_df['Area_280'] / valid_df['
    Total_Valid_Area'].replace(0, pd.NA)
valid_df['Relative_Proportion'] = valid_df['Relative_Proportion'].fillna(0.0)

# Step 3: Generate Comparative Analysis (Stacked Bar Chart)
# Feedback: Inefficient Pivot Table Construction for Large Datasets
pivot_df = valid_df.pivot_table(
    index='run_no',
    columns='Peak_ID',
    values='Relative_Proportion',
    aggfunc='sum'
)

# Sort runs for consistent plotting
pivot_df = pivot_df.sort_index()

if not pivot_df.empty:
    plt.figure(figsize=(12, 6))
    pivot_df.plot(kind='bar', stacked=True, figsize=(12, 6), colormap='tab20')
    plt.xlabel('run_no')
    plt.ylabel('Relative Proportion')
    plt.title('Relative Compound Proportions per Run')
    plt.ylim(0, 1.05)
    plt.legend(title='Peak_ID', bbox_to_anchor=(1.05, 1), loc='upper left')
    plt.tight_layout()
    # Feedback: Add os.makedirs to ensure directory exists
    os.makedirs(os.path.dirname(output_image), exist_ok=True)
    plt.savefig(output_image, dpi=150)
    plt.close()
```

```
else:
    logging.warning("No␣data␣to␣plot.␣Skipping␣chart␣generation.")

# Step 4: Identify Outliers
run_stats = valid_df.groupby('run_no').agg({
    'Total_Valid_Area': 'first',
    'Peak_ID': 'count',
    'Relative_Proportion': 'sum'
}).reset_index()

run_stats.columns = ['run_no', 'Total_Valid_Area', 'Number_of_Peaks', '
    Sum_of_Relative_Proportions']

# Feedback: Replace lambda with vectorized operation
outlier_condition = (run_stats['Total_Valid_Area'] == 0) | (abs(run_stats['
    Sum_of_Relative_Proportions'] - 1.0) > 0.05)
run_stats['Outlier_Flag'] = outlier_condition.map({True: 'Yes', False: 'No'})

# Add warning log regarding potential underestimation
logging.warning("Total_Valid_Area␣may␣be␣underestimated␣if␣small␣peaks␣were␣missed
    ␣by␣the␣fixed␣threshold␣in␣Step␣2.")

# Check for empty dataframe before generating summary
if run_stats.empty:
    print('No␣runs␣found␣to␣summarize.')
    os.makedirs(os.path.dirname(output_summary), exist_ok=True)
    with open(output_summary, 'w') as f:
        f.write("#␣Run-Level␣Normalization␣Summary\n\nNo␣runs␣found␣to␣summarize.\
            n")
else:
    # Feedback: Replace #summary_df.to_markdown()# with #summary_df.to_string()#
    summary_df = run_stats[['run_no', 'Total_Valid_Area', 'Number_of_Peaks', '
        Sum_of_Relative_Proportions', 'Outlier_Flag']].head(20)
    md_content = "#␣Run-Level␣Normalization␣Summary\n\n"
    md_content += summary_df.to_string(index=False)

    # Append to final_summary.md
    os.makedirs(os.path.dirname(output_summary), exist_ok=True)
    if os.path.exists(output_summary):
        mode = 'a'
        md_content = "\n\n" + md_content
    else:
        mode = 'w'

    with open(output_summary, mode) as f:
        f.write(md_content)

print("Analysis␣complete.␣Outputs␣saved␣to␣/workspace/")
```

Listing 3: Code_3